

ARK-455b Preregistration

ProofRecord Verification After Tampering — Corrected Retest with Validity-Window Semantics

Status: LOCKED (pre-execution)

Lock date: 2026-07-17

Series: ExecutionProof authorization-boundary corpus

Predecessor: ARK-455 (v1.0 executed FAIL; see root-cause erratum ../ark-455/CORRECTION.md)

Substrate: Classical software (local execution, no QPU)

Verifiers: Dual independent (V1 JavaScript, V2 Python)

LOCK INSTRUCTION

This preregistration is committed to version control BEFORE any arm is executed. The generator, both verifiers, both run scripts, and this document are hashed in MANIFEST.txt at lock time. Results are produced only after the lock commit. The recorded verdict — PASS, FAIL, or KILLED — stands as executed regardless of outcome. This is a direct discipline commitment: ARK-455b exists precisely because ARK-455 v1.0's harness carried an undetected defect, and this retest is designed so that class of defect cannot recur silently.

1. Preamble and Series Context

1.1 Why ARK-455b Exists

ARK-455 ("ProofRecord Verification After Tampering") executed on 2026 and recorded a **FAIL** verdict. Its own RESULTS.md honestly flagged the anomaly: Arm 3 (timestamp tampering) was rejected by **neither** verifier (0% rejection), while the other six tampering targets were detected at 100% and V1-V2 concordance was 100%.

A post-hoc code audit of the published artifact (Zenodo DOI 10.5281/zenodo.21408545) established the true root cause definitively. The ARK-455 generator's Arm-3 timestamp "tamper" was:

```
dt.replace(microsecond=(dt.microsecond + 1000000) % 1000000)
```

`(microsecond + 1000000) % 1000000 == microsecond` for every value of `microsecond`, and the operation never touches the seconds field. **The mutation was a mathematical no-op.** The "tampered" Arm-3 records were byte-identical to

untampered controls. Both verifiers therefore CORRECTLY accepted them: there was nothing to detect.

This is decisive on three points that had been open hypotheses:

1. The timestamp **is** included in the JCS-canonicalized, Ed25519-signed payload. (Both ARK-455 verifiers list it among the seven signed fields.)
2. Both verifiers **do** check the timestamp — a genuine change breaks the signature and is rejected.
3. The failure was **neither a verifier defect nor a specification gap**. It was a generator/test-harness defect: the tamper function did not tamper.

ARK-455's recorded FAIL verdict **stands** (it is not retracted; see the erratum).

What ARK-455b corrects is the ability of the harness to lie by omission, and it additionally hardens the specification with validity-window semantics that a bare signature check cannot cover.

Provenance note. ARK-455 executed a simplified 8-arm / 800-record variant rather than any larger design. Per the series discipline, the registry records which preregistration actually ran: ARK-455b likewise runs the concrete 9-arm / 900-record design specified in this document, and no other.

1.2 What ARK-455b Changes

#	Change	Rationale
1	Arm 3 timestamp tamper is a real +1 second mutation (via <code>timedelta</code>) that changes the signed bytes.	Eliminate the no-op; make post-signing timestamp tampering genuinely detectable.
2	New Arm 9 : timestamp is backdated beyond the validity window before signing, producing a valid signature over an expired record.	Test validity-window / expiry semantics that a bare signature check cannot catch (expiry per ARK-442).
3	Validity-window gate added to both verifiers (Gate B).	ACCEPT requires a valid signature AND an in-window timestamp.
4	Mutation-effectiveness gate in both run scripts.	Every REJECT-arm record must carry an effective mutation; a no-op aborts the run. A defect of the ARK-455 class can never again masquerade as a detection failure.
5	Portable, <code>__file__</code> -relative paths in all scripts.	Reproducibility outside the original absolute path.

1.3 The Doctrine Being Tested

Unchanged from ARK-455 in spirit, sharpened in scope:

An independent verifier, given the ProofRecord schema, the signature verification key, and the declared validity window, detects every tampering attempt AND every out-of-window (expired or future-dated) record, rejecting each while accepting only genuine, in-window originals.

ARK-455b tests both halves: cryptographic integrity (Gate A) and temporal validity (Gate B). The second half is the ExecutionProof position, established in ARK-442, that stale authority must fail closed — now enforced at the independent-verification layer, not only at the execution boundary.

2. Primary Hypothesis

H1 (Primary): An independent verifier, given the ProofRecord schema, the Ed25519 verification key, the verification-time instant, and the validity TTL, will:

- **ACCEPT** the original unaltered, in-window record (Arm 1).
- **REJECT** every post-signing tampered record (Arms 2–8, all seven fields).
- **REJECT** every pre-signing expired record whose signature is valid but whose timestamp is out of the validity window (Arm 9).

Operationally:

- **ACCEPT condition:** Arm 1 $\rightarrow V_accept_rate \geq 0.95$.
- **REJECT condition:** Arms 2–9 $\rightarrow V_reject_rate \geq 0.95$ (each arm).

H1 is confirmed if and only if all three primary pass criteria in Section 6 are met simultaneously.

3. Secondary Hypotheses

H2a — Tampering universality: Every post-signing tampering target (decision, timestamp, payload_hash, evidence_references, actor, execution_outcome, review_path) produces $V_reject \geq 0.95$. With the Arm-3 no-op removed, no target is exempt.

H2b — Dual verifier agreement: Two independent verifiers (V1 JavaScript, V2 Python), built from this prose specification with no shared source, agree with $\geq 99\%$ concordance across all arms.

H2c — Signature binding: Any post-signing field change breaks the Ed25519 signature; no tampered record in Arms 2–8 yields a valid signature.

H2d — Validity-window binding (new): A record whose timestamp is out of the declared window is REJECTED even when its signature is cryptographically valid (Arm 9). Temporal validity is enforced independently of signature validity.

H2e — Mutation effectiveness (new, methodological): Every record in every REJECT arm carries an effective mutation — a changed signed byte string (post-signing arms) or an out-of-window signed timestamp (pre-signing arm). This is asserted by the harness and gates execution.

4. Record Architecture

4.1 ProofRecord Schema

Seven signed fields (unchanged from ARK-455): `decision`, `timestamp`, `payload_hash`, `evidence_references`, `actor`, `execution_outcome`, `review_path`. Plus a `signature` field (Ed25519, 64 bytes / 128 hex chars). Full schema in `schemas/proofrecord_schema.json` (v1.1).

4.2 Timestamp Specification (sharpened)

The `timestamp` MUST be an RFC 3339 UTC instant carrying an explicit UTC designator (`Z` or `+00:00`), serialized inside the RFC 8785 (JCS) canonical form that is Ed25519-signed. Consequences:

- A post-signing edit to `timestamp` changes the signed bytes and breaks the signature (Gate A rejects).
- A pre-signing timestamp outside the validity window yields a valid signature but MUST be rejected by the validity-window check (Gate B rejects).

4.3 Signature Generation

Ed25519 over `canonicalize_jcs(7 signed fields)`. Deterministic key from a recorded 256-bit seed per arm.

4.4 Validity Window (new)

A ProofRecord is valid only within `[issuance_time, issuance_time + TTL]`. `VALIDITY_TTL_SECONDS = 300`. At verification time `T`, with record timestamp `t`:

- `age = T - t`.
- REJECT if `age < 0` (issued in the future).
- REJECT if `age > TTL` (expired).
- Otherwise the window check passes.

Expiry semantics follow ARK-442 (stale authority fails closed).

4.5 Verification Logic — Two Gates

Both gates must pass to ACCEPT:

- **Gate A (signature):** remove `signature`, canonicalize the 7 fields via JCS, verify Ed25519. Invalid → REJECT.
 - **Gate B (validity window):** parse the signature-verified `timestamp` as RFC 3339 UTC, apply Section 4.4. Out of window → REJECT.
-

5. Arm Specifications

100 records per arm, 9 arms, 900 records total.

Arm	Label	Mode	Expected verdict
1	ACCEPT-original	control (valid, in-window)	ACCEPT
2	REJECT-decision	post-signing tamper	REJECT (Gate A)
3	REJECT-timestamp-postsign	post-signing tamper (real +1s)	REJECT (Gate A)
4	REJECT-payload_hash	post-signing tamper	REJECT (Gate A)
5	REJECT-evidence_refs	post-signing tamper	REJECT (Gate A)
6	REJECT-actor	post-signing tamper	REJECT (Gate A)
7	REJECT-outcome	post-signing tamper	REJECT (Gate A)
8	REJECT-review_path	post-signing tamper	REJECT (Gate A)
9	REJECT-timestamp-presign-expired	pre-signing backdate (valid sig, expired)	REJECT (Gate B)

5.1 Record Generation Procedure

Per arm: draw a 256-bit seed, derive the Ed25519 key, generate 100 base records, sign, then (for tamper arms) apply the arm's mutation. `generate_arm_records` returns `(records, audit)`; `audit[i].mutation_effective` is `True` iff the mutation is effective (post-signing byte change, or pre-signing out-of-window timestamp), `None` for the control arm.

5.2 Mutation-Effectiveness Gate

Before scoring, both run scripts verify that every REJECT-arm record has `mutation_effective == True`. If any record fails, the run **ABORTS** — a no-op tamper can never again be silently scored as a detection failure. This is the methodological core of ARK-455b.

6. Pass / Fail / Killed Criteria (Primary — Preregistered)

Let, taking the worst of V1 and V2 at each point:

- `V_accept_original` = Arm 1 accept rate.
- `V_reject_min` = minimum reject rate over Arms 2-9.
- `V_margin` = `V_reject_min - (1 - V_accept_original)`.

PASS iff all three:

- **C1:** `V_accept_original` ≥ 0.95
- **C2:** `V_reject_min` ≥ 0.95
- **C3:** `V_margin` ≥ 0.85

KILLED if the kill-gate fails (see Section 7) — execution does not proceed.

FAIL if the kill-gate passes but any of C1–C3 is not met. A FAIL is recorded and published as executed.

Verifier agreement requirement: V1–V2 concordance $\geq 99\%$ overall; a lower concordance is itself reported as a finding.

7. Kill-Gate Calibration

Before arm execution, generate 100 records (50 valid Arm-1, 50 tampered cycling Arms 2–9). Require: (a) mutation-effectiveness 100% on the 50 tampered; (b) both verifiers ACCEPT all 50 valid (sanity); (c) V1–V2 concordance $\geq 99\%$. Any failure aborts before arms run.

8. Verifier Independence Rule

V1 (JavaScript) and V2 (Python) are implemented solely from this prose specification. Neither references the other's source, nor the generator. Each carries an isolation notice in its header.

9. Execution Plan (Strict Sequence)

1. Commit this preregistration + code + MANIFEST.txt hashes (LOCK).
2. Run `run_killgate.py`. If not PASS $\rightarrow$ record KILLED, stop.
3. Run `run_arms.py` (aborts on any ineffective mutation).
4. Record actual per-arm results, overall metrics, and verdict in RESULTS.md.
5. Commit results. Push branch, open PR for review.

10. Honest Boundary Statements

- ARK-455b is a **classical software** test. It makes no quantum claim.
- It validates the ProofRecord verification model and validity-window semantics; it does not claim coverage of multi-field tampering, replay across keys, or adversarial signature forgery — those remain deferred.
- The expected outcome is PASS, but the verdict is whatever the run produces. If it FAILs, the FAIL is published.

- ARK-455b does not overturn ARK-455's recorded FAIL. It corrects the cause (harness no-op) and extends the specification (validity window).
-

11. Connection to Corpus

ARK-455b completes the audit-integrity dimension opened by ARK-455 and joins it to the temporal dimension established by ARK-442: an independent auditor rejects both tampered and stale ProofRecords. It also introduces a reusable methodological safeguard — the mutation-effectiveness gate — for every future tamper-style testbed.

12. MANIFEST

See MANIFEST.txt for SHA-256 hashes of all locked files, written at lock time.